TP2 - Intro à la statistique spatiale 2A - sf - corrigé

OBJECTIFS DU TP :

- Le but de ce TP est de se familiariser à la manipulation d'objets spaciaux, à l'aide du package `sf`. Vous trouverez un cheatsheet ici
- Afin d'utiliser une version de R plus récente (et une version du package sf plus récente aussi), vous travaillerez sur le datalab (plateforme du sspcloud, service de l'Insee) : https://datalab.sspcloud.fr. . Si ce n'est déjà fait, il vous faudra créer un compte utilisateur puis créer un service Rstudio.
- Si besoin (ce ne sera pas le cas pour cette séance), les fonds disponibles sous "U:/Eleves/Cartographie/Fonds_carte". **PS : Ce répertoire ne doit JAMAIS être votre répertoire de travail !** Il s'agit d'un répertoire où l'on met à disposition des fonds de carte au service de tout le monde. Leur modification pénaliserait donc tous ses utilisateurs.

Exercice 1

0. Commencer par créer votre répo github "tp_stats_spatiales". Créer un service Rstudio sur le datalab en saisissant le lien https de votre repo Git (suivre la procédure "Github et le datalab" si besoin). Une fois le service ouvert, créer un projet qui pointe vers votre dossier intitulé "tp_stats_spatiales" (qui est normalement déjà présent à l'ouverture du service).

Charger ensuite les package sf et dplyr.

```r
# Chargement des packages
library(sf)
library(dplyr)
```

1. Importer le fond communal "commune_francemetro_2021.gpkg" disponible dans le dossier "France_metro". Pour cela vous utiliserez la fonction `st_read` du package `sf`. Quelles informations apparaissent dans la console ?

```r
commune_francemetro_2021 <- st_read("commune_francemetro_2021.gpkg")
```

```
Reading layer `commune_francemetro_2021' from data source
  `/home/onyxia/work/2A_stat_spatiale/2025-2026/TP2/commune_francemetro_2021.gpkg'
  using driver `GPKG'
Simple feature collection with 34836 features and 16 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: 99225.97 ymin: 6049647 xmax: 1242375 ymax: 7110480
Projected CRS: RGF93 v1 / Lambert-93
```

```r
# Renseigne la geometry (Multypolygon)
# dimension: XY fait référence à un système de coordonnées dans un espace euclidien
# Bounding box: désigne les coordonnées du cadre contenant/entourant nos objets spaciaux
# Projected CRS: désigne le système de projection à travers son code EPSG
```

2. Faites un résumé/descriptif du contenu de l'objet importé, comme vous le feriez pour un dataframe.

```r
str(commune_francemetro_2021)
```

```
Classes 'sf' and 'data.frame':  34836 obs. of  17 variables:
 $ code      : chr  "07038" "07055" "07304" "18151" ...
 $ libelle   : chr  "Borne" "Charmes-sur-Rhône" "Salavas" "Montigny" ...
```

```
 $ cheflieu  : chr  "0" "0" "0" "0" ...
 $ tncc      : chr  "0" "0" "0" "0" ...
 $ dct       : chr  "0713" "0717" "0715" "1814" ...
 $ dar       : chr  "071" "073" "071" "181" ...
 $ z20       : chr  "8402" "8431" "8402" "2402" ...
 $ duu20     : chr  NA "07203" "07106" NA ...
 $ aav20     : chr  NA "049" NA NA ...
 $ typo_aav20: chr  NA "20" NA NA ...
 $ epc       : chr  "200072007" "200041366" "200039808" "200066330" ...
 $ bv        : chr  "48080" "26362" "07330" "18003" ...
 $ cdcte     : chr  "07D" "07D" "07D" "18D" ...
 $ dep       : chr  "07" "07" "07" "18" ...
 $ reg       : chr  "84" "84" "84" "24" ...
 $ surf      : num  32.2 5.9 17.2 29.1 19.2 ...
 $ geom      :sfc_MULTIPOLYGON of length 34836; first list element: List of 1
  ..$ :List of 1
  .. ..$ : num [1:31, 1:2] 781948 781212 780376 780044 778763 ...
  ..- attr(*, "class")= chr [1:3] "XY" "MULTIPOLYGON" "sfg"
 - attr(*, "sf_column")= chr "geom"
 - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..: NA NA NA NA NA NA NA NA NA NA ...
  ..- attr(*, "names")= chr [1:16] "code" "libelle" "cheflieu" "tncc" ...
```

```r
# L'information importante est le fait que notre objet soit un objet sf ET un dataframe
```

3. Afficher maintenant les dix premières lignes de la table et regarder la dernière colonne.

```r
head(commune_francemetro_2021,10)
```

```
Simple feature collection with 10 features and 16 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: 640088 ymin: 6364325 xmax: 846079 ymax: 6687546
Projected CRS: RGF93 v1 / Lambert-93
    code           libelle cheflieu tncc  dct dar  z20 duu20 aav20 typo_aav20
1  07038             Borne        0    0 0713 071 8402  <NA>  <NA>       <NA>
2  07055 Charmes-sur-Rhône        0    0 0717 073 8431 07203   049         20
3  07304           Salavas        0    0 0715 071 8402 07106  <NA>       <NA>
4  18151          Montigny        0    0 1814 181 2402  <NA>  <NA>       <NA>
5  18152         Montlouis        0    0 1808 182 2402  <NA>  <NA>       <NA>
6  18153            Morlac        0    0 1808 182 2402  <NA>   300         20
7  18154      Mornay-Berry        0    0 1810 182 0060  <NA>  <NA>       <NA>
8  18155 Mornay-sur-Allier        0    0 1809 182 0060  <NA>   097         20
9  18156          Morogues        0    0 1814 181 2402  <NA>   064         20
10 18157       Morthomiers        0    0 1807 181 2402 18501   064         20
         epc    bv cdcte dep reg  surf                           geom
1  200072007 48080   07D  07  84 32.19 MULTIPOLYGON (((781948 6386...
2  200041366 26362   07D  07  84  5.90 MULTIPOLYGON (((845215 6417...
3  200039808 07330   07D  07  84 17.17 MULTIPOLYGON (((810305 6364...
4  200066330 18003   18D  18  24 29.14 MULTIPOLYGON (((675141 6677...
5  200027076 36046   18D  18  24 19.25 MULTIPOLYGON (((642892 6630...
6  200049484 18197   18D  18  24 33.27 MULTIPOLYGON (((647636 6621...
7  200007177 18033   18D  18  24  9.43 MULTIPOLYGON (((690655 6658...
8  241800432 18242   18D  18  24 22.16 MULTIPOLYGON (((702446 6632...
9  200066330 18003   18D  18  24 31.01 MULTIPOLYGON (((671365 6680...
10 241800507 18207   18D  18  24 14.75 MULTIPOLYGON (((644042 6657...
```

```r
# La dernière colonne contient la géométry, c'est à dire les coordonnées géographiques
```

4. Afficher le système de projection de la table en utilisant la fonction `st_crs`.

```r
st_crs(commune_francemetro_2021)
```

```
Coordinate Reference System:
  User input: RGF93 v1 / Lambert-93
  wkt:
```

```
PROJCRS["RGF93 v1 / Lambert-93",
    BASEGEOGCRS["RGF93 v1",
        DATUM["Reseau Geodesique Francais 1993 v1",
            ELLIPSOID["GRS 1980",6378137,298.257222101,
                LENGTHUNIT["metre",1]]],
        PRIMEM["Greenwich",0,
            ANGLEUNIT["degree",0.0174532925199433]],
        ID["EPSG",4171]],
    CONVERSION["Lambert-93",
        METHOD["Lambert Conic Conformal (2SP)",
            ID["EPSG",9802]],
        PARAMETER["Latitude of false origin",46.5,
            ANGLEUNIT["degree",0.0174532925199433],
            ID["EPSG",8821]],
        PARAMETER["Longitude of false origin",3,
            ANGLEUNIT["degree",0.0174532925199433],
            ID["EPSG",8822]],
        PARAMETER["Latitude of 1st standard parallel",49,
            ANGLEUNIT["degree",0.0174532925199433],
            ID["EPSG",8823]],
        PARAMETER["Latitude of 2nd standard parallel",44,
            ANGLEUNIT["degree",0.0174532925199433],
            ID["EPSG",8824]],
        PARAMETER["Easting at false origin",700000,
            LENGTHUNIT["metre",1],
            ID["EPSG",8826]],
        PARAMETER["Northing at false origin",6600000,
            LENGTHUNIT["metre",1],
            ID["EPSG",8827]]],
    CS[Cartesian,2],
        AXIS["easting (X)",east,
            ORDER[1],
            LENGTHUNIT["metre",1]],
        AXIS["northing (Y)",north,
            ORDER[2],
            LENGTHUNIT["metre",1]],
    USAGE[
        SCOPE["Engineering survey, topographic mapping."],
        AREA["France - onshore and offshore, mainland and Corsica (France métropolitaine including C
        BBOX[41.15,-9.86,51.56,10.38]],
    ID["EPSG",2154]]
```

```r
# La fonction décrit tout un tas d'information sur le système de projection,
#   son libellé et sa mesure (UNIT)
```

5. Créer une table "communes_Bretagne" ne contenant que les communes bretonnes. Ne conserver que les colonnes (code, libelle, epc, dep, surf) en utilisant la fonction `select()` de `dplyr`. Votre table contient-elle uniquement les 5 variables sélectionnées ?

```r
communes_Bretagne <- commune_francemetro_2021 %>%
  filter(reg=="53") %>%
  select(code, libelle, epc, dep, surf)

# La variable de Geometry qui caractérise l'objet sf, n'a pas été supprimée.
```

6. Assurez-vous que cette nouvelle table est toujours un objet `sf`.

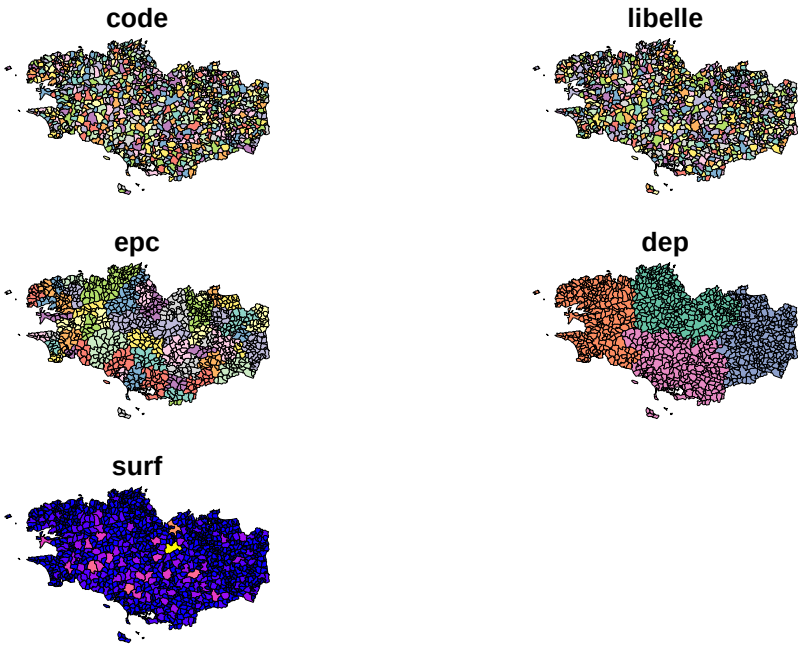
```r
str(communes_Bretagne)
```

```
Classes 'sf' and 'data.frame':  1208 obs. of  6 variables:
 $ code   : chr  "22001" "22002" "22003" "22004" ...
 $ libelle: chr  "Allineuc" "Andel" "Aucaleuc" "Bégard" ...
 $ epc    : chr  "200067460" "200069391" "200068989" "200067981" ...
 $ dep    : chr  "22" "22" "22" "22" ...
```

```
 $ surf   : num  24.39 12.41 6.54 36.37 14.34 ...
 $ geom   :sfc_MULTIPOLYGON of length 1208; first list element: List of 1
  ..$ :List of 1
  .. ..$ : num [1:18, 1:2] 264501 263540 262806 262082 262589 ...
  ..- attr(*, "class")= chr [1:3] "XY" "MULTIPOLYGON" "sfg"
 - attr(*, "sf_column")= chr "geom"
 - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",..: NA NA NA NA NA
  ..- attr(*, "names")= chr [1:5] "code" "libelle" "epc" "dep" ...
```

7. Appliquer la fonction `plot` sur votre table. (Indice : l'argument `lwd` vous permet de jouer sur l'épaisseur des lignes).

```r
plot(communes_Bretagne, lwd = 0.1)
```



8. Faire la question précédente en utilisant la fonction `st_geometry()` dans votre plot.

```r
plot(st_geometry(communes_Bretagne), lwd = 0.5)
```

9. Créer une variable de surface appelée "surf2" en utilisant les fonctions `st_area()` sur votre variable de geometry. En quelle unité est la variable créée ?

```r
communes_Bretagne<-communes_Bretagne %>%
  mutate(surf2=st_area(geom))

str(communes_Bretagne$surf2)
```

```
 Units: [m^2] num [1:1208] 24537548 12018097 6249850 36630972 14001048 ...
```

```r
# Variable en m²
```

10. Modifier la variable créée pour la convertir en km².

```r
# Cela revient à diviser notre variable par 1000*1000. Une façon plus élégante est
#   d'utiliser le convertisseur d'unités issu du package units : set_units()

communes_Bretagne <- communes_Bretagne %>%
  mutate(surf2 = units::set_units(surf2, km*km))

str(communes_Bretagne$surf2)
```

```
 Units: [km^2] num [1:1208] 24.54 12.02 6.25 36.63 14 ...
```

11. Les variables `surf` et `surf2` sont-elles égales ? Pourquoi selon vous ?

```r
# Plusieurs raisons possibles :
# - Calcul sur des systèmes de projection différents (ce n'est pas le cas ici)
# - Calcul sur des fonds différents : les fonds n'ont pas les mêmes précisions de contours.
# - On trouve des fonds "simplifiés" des communes et des fonds détaillés (en général
#    de la BDTOPO). Les fonds détaillés sont + précis et donc beaucoup plus lourds.
```

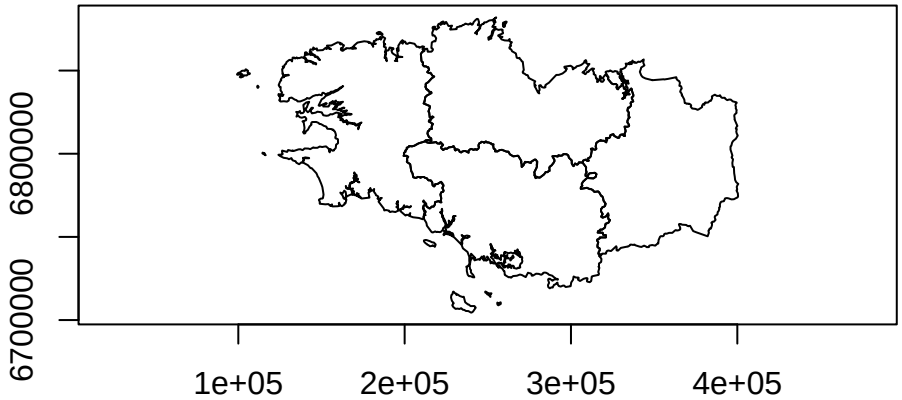
12. L'objectif est de créer une table départementale "dept_bretagne" sans doublons. Cette table devra contenir le code departement et la superficie du département. Représenter le nouveau fond sur une carte avec la fonction `plot()`.

```r
# Avec la fonction summarise
dept_bretagne2<-communes_Bretagne %>%
  group_by(dep) %>%
  summarise(surf=sum(surf))

plot(st_geometry(dept_bretagne2))
```

```r
# Les géométries ont été regroupées pour faire une geometry par département.
```

13. Constituer cette fois un fond départemental en utilisant les fonctions `summarise()` et `st_union()`. À la différence de la table précedemment créée, le fond ne contiendra que le code dept et la geometry (aucune variable numérique ne sera utilisée). Faire ensuite un plot de votre table pour vérifier que les geometry ont bien été regroupés par département.

```
dept_bretagne<-communes_Bretagne %>%
  group_by(dep) %>%
  summarise(geometry=st_union(geom))

plot(st_geometry(dept_bretagne), axes=TRUE)
```




```
# c'est la méthode à préconiser pour regrouper des géométries - rien n'interdit de faire
#   un summarise mais on n'a pas toujours une variable numérique à disposition.
```

14. Créer une table "centroid_dept_bretagne" contenant les centroïdes des départements bretons.

    a. Quel est le type de géometrie associé à ces centroïdes ?

```
centroid_dept_bret<-st_centroid(dept_bretagne)
```

```
Warning: st_centroid assumes attributes are constant over geometries
```

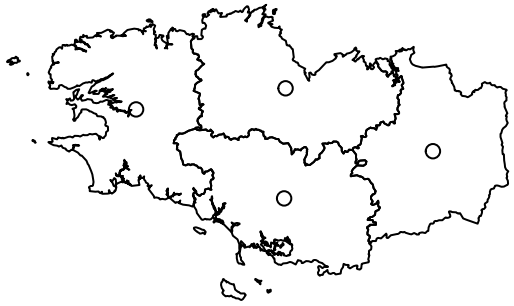
```
class(centroid_dept_bret$geometry)
```

```
[1] "sfc_POINT" "sfc"
```

```
# Type point
```

    b. Représenter les départements bretons et leurs centroïdes sur une même carte, avec deux appels à la fonction `plot()` et en ajoutant l'argument `add = TRUE` sur le second appel.

```
plot(st_geometry(dept_bretagne))
plot(st_geometry(centroid_dept_bret), add = TRUE)
```



c. Ajouter le nom du départment dans le fond de centroïdes. La variable aura pour nom `dept_lib`. Plusieurs solutions sont possibles, la plus propre étant d'utiliser une petite table de passage et de la fusionner avec le fond de centroïdes.

```
dept_lib <- tibble(
  dep = c("22","29","35","56"),
  dep_lib = c("Côtes d'Armor","Finistère","Ille-et-Vilaine","Morbihan")
)
centroid_dept_bret <- centroid_dept_bret %>%
  left_join(
    dept_lib,
    by = "dep"
  )
```

d. Récupérer les coordonnées des centroïdes dans un data.frame appelé `centroid_coords` avec la fonction `st_coordinates()`. Observer l'objet obtenu. Ajouter ensuite les colonnes `dep` et `dep_lib` du fond de `centroid_dept_bret` avec la fonction `bind_cols()`. Vous ferez attention à ce que `centroid_coords` ne contienne pas de géométrie (Indice : utiliser `st_drop_geometry()`).

```
centroid_coords <- st_coordinates(centroid_dept_bret)
centroid_coords %>% str() ## On obtient un df avec seulement deux colonnes X et Y
```

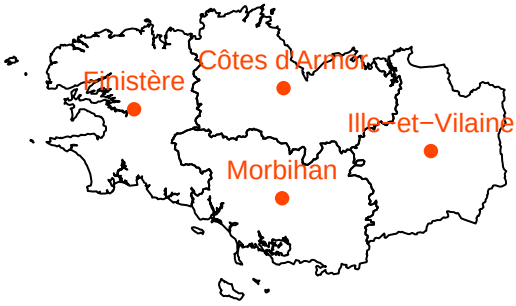
```
 num [1:4, 1:2] 266688 176829 355270 265826 6831745 ...
 - attr(*, "dimnames")=List of 2
  ..$ : NULL
  ..$ : chr [1:2] "X" "Y"
```

```
centroid_coords <- centroid_coords %>%
  bind_cols(
    centroid_dept_bret %>%
      select(dep, dep_lib) %>%
      st_drop_geometry()
  )
centroid_coords %>% str()
```

```
tibble [4 x 4] (S3: tbl_df/tbl/data.frame)
 $ X      : num [1:4] 266688 176829 355270 265826
 $ Y      : num [1:4] 6831745 6819054 6793927 6765548
 $ dep    : chr [1:4] "22" "29" "35" "56"
 $ dep_lib: chr [1:4] "Côtes d'Armor" "Finistère" "Ille-et-Vilaine" "Morbihan"
```

e. Représenter les départements, leur centroïde (comme en 14.b) et leur nom (avec la fonction `text()`) sur une seule carte.

```r
plot(st_geometry(dept_bretagne))
plot(st_geometry(centroid_dept_bret), pch = 16, col = "orangered", add = TRUE)
text(
  x = centroid_coords$X,
  y = centroid_coords$Y,
  labels = centroid_coords$dep_lib,
  pos = 3,
  cex = 0.8,
  col = "orangered"
)
```



15. À l'aide de la fonction `st_intersects()`, retrouver dans quelle commune se situe le centroïde de chaque département breton.

```r
commune_centroid_bret <- st_intersects(communes_Bretagne, centroid_dept_bret)
typeof(commune_centroid_bret)
```

```
[1] "list"
```

```r
# Il s'agit d'une liste de longueur égale au nombre d'éléments de mon 1er paramètre
#   (ici cela correspond à mon nombre de communes).

# La fonction intersects renvoie une liste contenant autant d'éléments que d'observations
# dans notre table communale. Lorsque qu'il y a intersection l'élément est renseigné, vide sinon.

# On récupère les éléments non vides de notre liste avec la commande suivante
which(lengths(commune_centroid_bret)>0)
```

```
[1]  148  476  647 1092
```

```r
# Pour avoir les communes concernées, on applique ce vecteur de position à notre fichier de communes
commune_centre_dep<-communes_Bretagne[which(lengths(commune_centroid_bret)>0),]

# Equivalent en écriture dplyr
commune_centre_dep <- communes_Bretagne %>%
  filter(lengths(commune_centroid_bret) > 0)


# REMARQUE, il est ici plus simple de faire l'intersection dans l'autre sens,
#   en mettant les centroid en 1er paramètre
commune_centroid_bret2 <- st_intersects(centroid_dept_bret, communes_Bretagne)
# Cela renvoie une liste de 4 éléments puis de faire
commune_centre_dep<-communes_Bretagne[unlist(commune_centroid_bret2),]
```

```r
# Une autre façon est d'utiliser les crochets et l'option op=st_intersects
communes_Bretagne[centroid_dept_bret,,op=st_intersects]
```

```
Simple feature collection with 4 features and 6 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: 176045 ymin: 6762278 xmax: 358530.1 ymax: 6835082
Projected CRS: RGF93 v1 / Lambert-93
      code      libelle       epc dep  surf                           geom
148  22170 Plaine-Haute 200069409  22 15.91 MULTIPOLYGON (((267187 6829...
476  29139      Lopérec 200067197  29 40.03 MULTIPOLYGON (((177715 6818...
647  35024       Betton 243500139  35 26.80 MULTIPOLYGON (((356273 6793...
1092 56141  Moustoir-Ac 200067221  56 33.93 MULTIPOLYGON (((262049 6762...
               surf2
148  15.44731 [km^2]
476  40.61583 [km^2]
647  26.69068 [km^2]
1092 33.83409 [km^2]
```

16. Faire la même question avec la fonction `st_intersection()` puis avec la fonction `st_within()`. Quelles différences voyez-vous avec la fonction `st_intersects()` ?

```r
Q16_intersection <- st_intersection(communes_Bretagne,centroid_dept_bret)
```

```
Warning: attribute variables are assumed to be spatially constant throughout
all geometries
```

```r
# La fonction st_intersection nous renvoit directement le résultat attendu (dataframe).
# Cette fonction est donc beaucoup plus pratique. Cependant, dans le cas de calculs lourds,
#  elle peut être très longue.

# Avec la fonction within, on regarde si un objet (1er paramètre) est contenu dans un
#   autre objet (2e paramètre). L'ordre des paramètre a donc une importance.
#   Cette fonction renvoie encore une liste.
Q16_within <- st_within(centroid_dept_bret,communes_Bretagne)
```

17. Calculer la distance séparant les centroïdes des départements et leur chefs-lieux. Les chefs-lieux des départements bretons sont les communes de Saint-Brieuc (22), Quimper (29), Rennes (35) et Vannes (56). Vous utiliserez la fonction `st_distance()`.

```r
Q17 <- st_distance(
  centroid_dept_bret,
  communes_Bretagne %>%
    filter(libelle %in% c("Saint-Brieuc","Quimper","Rennes","Vannes"))
)

Q17 <- data.frame(Q17)
rownames(Q17) <- centroid_dept_bret$dep
colnames(Q17) <- c("Saint-Brieuc","Quimper","Rennes","Vannes")
```

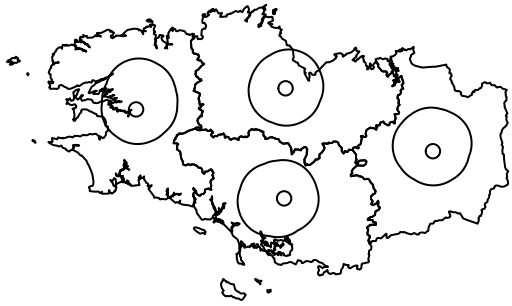
18. Quelles sont les communes à moins de 20 km (à vol d'oiseau) de chaque centroïde ?

    a. Utilisez la fonction `st_buffer(x,dist)` pour créer une zone de 20 km autour des centroïdes. Vous ferez attention aux unités.

```r
buffer <- st_buffer(commune_centre_dep, dist = 20000)
```

    b. Représenter sur une carte la géométrie obtenue avec la fonction `plot()`.

```r
plot(st_geometry(dept_bretagne))
plot(st_geometry(centroid_dept_bret),add=TRUE)
plot(buffer %>% st_geometry(),add=TRUE)
```



c. Récupérez les communes comprises dans les tampons obtenus au 18.a avec la fonction `st_intersection()`.

```
Q18 <- buffer %>%
  rename(code_centroid=code, dep_centroid=dep) %>%
  st_intersection(
    communes_Bretagne %>% select(code_intersect=code, dep_intersect=dep)
  )
```

```
Warning: attribute variables are assumed to be spatially constant throughout
all geometries
```

d. Combien de communes sont concernées par département ?

```
Q18 %>%
  st_drop_geometry() %>%
  filter(dep_centroid==dep_intersect) %>%
  group_by(dep_centroid) %>%
  count()
```

```
# A tibble: 4 x 2
# Groups:   dep_centroid [4]
  dep_centroid     n
  <chr>        <int>
1 22              93
2 29              88
3 35             113
4 56              71
```

19.a. Changer le système de projection des communes bretonnes pour le mettre en WGS84 (EPSG=4326), avec la fonction `st_transform()`.

```
communes_Bretagne_wgs84 <- st_transform(communes_Bretagne, 4326)
```

b. Représentez le fond ainsi produit.

```
par(mfrow = c(1,2))
plot(st_geometry(communes_Bretagne), main = "epsg:2154")
plot(st_geometry(communes_Bretagne_wgs84), main = "epsg:4326")
```

20. Recalculer l'aire des communes pour créer une variable `surf3`. Que se passe t'il ?

```
communes_Bretagne_wgs84 <- communes_Bretagne_wgs84 %>%
  mutate(surf3 = st_area(geom))

st_crs(communes_Bretagne_wgs84)
```

```
Coordinate Reference System:
  User input: EPSG:4326
  wkt:
GEOGCRS["WGS 84",
    ENSEMBLE["World Geodetic System 1984 ensemble",
        MEMBER["World Geodetic System 1984 (Transit)"],
        MEMBER["World Geodetic System 1984 (G730)"],
        MEMBER["World Geodetic System 1984 (G873)"],
        MEMBER["World Geodetic System 1984 (G1150)"],
        MEMBER["World Geodetic System 1984 (G1674)"],
        MEMBER["World Geodetic System 1984 (G1762)"],
        MEMBER["World Geodetic System 1984 (G2139)"],
        ELLIPSOID["WGS 84",6378137,298.257223563,
            LENGTHUNIT["metre",1]],
        ENSEMBLEACCURACY[2.0]],
    PRIMEM["Greenwich",0,
        ANGLEUNIT["degree",0.0174532925199433]],
    CS[ellipsoidal,2],
        AXIS["geodetic latitude (Lat)",north,
            ORDER[1],
            ANGLEUNIT["degree",0.0174532925199433]],
        AXIS["geodetic longitude (Lon)",east,
            ORDER[2],
            ANGLEUNIT["degree",0.0174532925199433]],
    USAGE[
        SCOPE["Horizontal component of 3D system."],
        AREA["World."],
        BBOX[-90,-180,90,180]],
    ID["EPSG",4326]]
```

```
# L'unité est en degré, le calcul des distances, aires ne peut se faire directement
#   sur d'anciennes versions de sf.
# avec sf 1.0.6 : le calcul se fait et les valeurs sont proches de surf2 sans être égales
```